

ANSH THAPA**B.TECH CSE**

Run all cells in notebook

CU24250224**Experiment No. 8**

Application of Optical Flow for Real-Time Object Tracking and Motion Analysis

Aim

To implement an optical flow-based motion analysis system for real-time object tracking and evaluate its effectiveness in detecting and tracking moving objects in video sequences using Python and OpenCV.

Purpose

While optical flow estimates the motion between consecutive video frames, its true potential lies in real-world applications such as object tracking, traffic monitoring, human activity recognition, autonomous navigation, video surveillance, and sports analytics. By analyzing motion vectors, computer vision systems can continuously track moving objects without relying solely on object detection algorithms.

In this experiment, students will develop a real-time object tracking application using optical flow techniques and analyze object trajectories, motion direction, and movement patterns in video sequences.

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Understanding of Optical Flow algorithms (Lucas-Kanade and Farneback)
- Jupyter Notebook, Google Colab, or Visual Studio Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
from IPython.display import clear_output
import time

# Load video
video_path = "/content/20090-307115630_medium.mp4"

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video loaded successfully")

# Get video information
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

print("FPS:", fps)
print("Total Frames:", total_frames)
```

```
Video loaded successfully
FPS: 25.0
Total Frames: 372
```

```
# Read first frame
ret, old_frame = cap.read()

if not ret:
    print("Could not read video")
else:

    # Convert frame to grayscale
    old_gray = cv2.cvtColor(
```

```

    old_frame,
    cv2.COLOR_BGR2GRAY
)

# Shi-Tomasi parameters
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

# Detect feature points
corners = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

print("Feature points detected:", len(corners))

# Draw detected points
feature_frame = old_frame.copy()

for point in corners:
    x, y = point.ravel()

    cv2.circle(
        feature_frame,
        (int(x), int(y)),
        5,
        (0, 255, 0),
        -1
    )

cv2.imshow('feature_frame')

```

Feature points detected: 100



```

# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15),

```

```

maxLevel=2,
criteria=(
    cv2.TERM_CRITERIA_EPS |
    cv2.TERM_CRITERIA_COUNT,
    10,
    0.03
)
)
)

# Generate random colors for trajectories
color = np.random.randint(
    0, 255,
    (100, 3)
)

# Create trajectory mask
mask = np.zeros_like(old_frame)

frame_count = 0

print("Starting Lucas-Kanade tracking...")

while True:

    ret, frame = cap.read()

    if not ret:
        break

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    # Calculate optical flow
    new_points, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        corners,
        None,
        **lk_params
    )

    if new_points is None:
        break

    # Select successfully tracked points
    good_new = new_points[status == 1]
    good_old = corners[status == 1]

    # Draw trajectories
    for i, (new, old) in enumerate(
        zip(good_new, good_old)
    ):

        a, b = new.ravel()
        c, d = old.ravel()

        a, b = int(a), int(b)
        c, d = int(c), int(d)

        # Draw trajectory
        mask = cv2.line(
            mask,
            (a, b),
            (c, d),
            color[i].tolist(),
            2
        )

        # Draw current point
        frame = cv2.circle(
            frame,
            (a, b),
            5,
            color[i].tolist(),
            -1
        )

    # Combine frame and trajectory
    output = cv2.add(
        frame,
        mask

```

```

)

# Display only every 5th frame
if frame_count % 5 == 0:

    clear_output(wait=True)
    # Run all cells in notebook

    cv2.putText(
        output,
        "Lucas-Kanade Optical Flow",
        (20, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (0, 255, 0),
        2
    )

    cv2.imshow(output)

# Update previous frame and points
old_gray = frame_gray.copy()

corners = good_new.reshape(
    -1,
    1,
    2
)

frame_count += 1

# Stop after 100 frames for faster execution
if frame_count >= 100:
    break

print("Lucas-Kanade tracking completed.")

```



Lucas-Kanade tracking completed.

1. How does object tracking differ from object detection?

Object Detection is the process of identifying and locating objects in an individual image or video frame. It determines the class of an object and its position, usually using a bounding box.

Object Tracking is the process of continuously following the same object across multiple video frames and estimating its position over time.

Run all cells in notebook

Thus, **detection identifies the object, whereas tracking follows the object's movement.**

2. Explain how Optical Flow can be used for real-time object tracking.

Optical Flow is used to estimate the motion of pixels or feature points between two consecutive video frames. It provides the **direction and magnitude of motion**.

For real-time object tracking:

1. Important feature points are detected in the current frame.
2. These points are tracked in the next frame using Optical Flow.
3. The displacement of the points is calculated.
4. The object's new position is estimated from these movements.
5. This process is repeated for every frame.

The **Lucas-Kanade Optical Flow algorithm** is commonly used for tracking feature points in real-time video.

3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Shi-Tomasi Corner Detection is used to identify strong and distinctive corner points in an image. These points contain useful intensity information and can be tracked reliably between consecutive frames.

In the **Lucas-Kanade Optical Flow algorithm**, Shi-Tomasi is generally used to select the feature points that need to be tracked. Lucas-Kanade then calculates the displacement of these points.

Therefore, **Shi-Tomasi detects suitable feature points, while Lucas-Kanade estimates their motion.**

4. Why is Optical Flow suitable for motion analysis in videos?

Optical Flow is suitable for video motion analysis because it provides information about the **direction and speed of apparent motion** between consecutive frames.

Its advantages include:

- Detecting moving objects.
- Estimating the direction of movement.
- Estimating relative motion or velocity.
- Tracking feature points across frames.
- Supporting real-time video processing.

Therefore, Optical Flow is widely used in **object tracking, surveillance, robotics, autonomous vehicles, and activity recognition.**

5. What challenges arise while tracking fast-moving or partially occluded objects?

Several challenges can occur during object tracking:

- **Fast motion:** Large displacement between consecutive frames can make feature tracking difficult.
- **Motion blur:** Rapid movement can blur object features.
- **Partial occlusion:** Some parts of an object may be hidden by other objects.
- **Lighting changes:** Changes in illumination can affect feature detection.
- **Background movement:** A moving background or camera can create additional optical flow.
- **Feature loss:** Important feature points may disappear from the frame.

These problems can reduce tracking accuracy and may cause the tracker to lose the object.

6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Optical Flow-Based Tracking	Deep Learning-Based Tracking
Estimates motion of pixels or feature points.	Uses neural networks to detect and track objects.
Usually requires less computational power.	Generally requires higher computational resources.
Can work without large training datasets.	Usually depends on trained or pre-trained models.
Effective for simple and smooth motion.	Can handle complex object appearance and motion.
Often faster for lightweight applications.	Can provide stronger object recognition and tracking capabilities.

Thus, **Optical Flow is useful for lightweight motion-based tracking, while deep learning methods are generally more capable of handling complex tracking situations.**

7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

Optical Flow can be used to estimate the **movement and relative motion of vehicles, pedestrians, and other objects**.

In traffic monitoring, it can be used for:

- Vehicle movement detection Run all cells in notebook
- Traffic flow estimation.
- Vehicle speed estimation.
- Detection of congestion.
- Pedestrian movement analysis.

In autonomous driving, Optical Flow can help estimate the motion of nearby objects, detect approaching obstacles, and provide information useful for **navigation and collision avoidance**.

8. Explain the effect of camera motion on Optical Flow estimation.

Camera motion can produce Optical Flow even when objects in the scene are stationary. For example, when the camera moves forward, stationary background objects appear to move relative to the camera.

As a result, the estimated Optical Flow contains both:

1. **Object motion**, and
2. **Camera-induced motion**.

This makes it difficult to determine whether a particular motion is caused by the object or by the camera.

To overcome this problem, techniques such as **camera-motion estimation, image stabilization, and motion compensation** can be used.

9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

Five real-world applications of Optical Flow are:

1. **Traffic monitoring** – analyzing vehicle movement and traffic flow.
 2. **Autonomous vehicles** – estimating the motion of vehicles and obstacles.
 3. **Video surveillance** – tracking people and detecting unusual movements.
 4. **Human activity recognition** – recognizing activities such as walking, running, and falling.
 5. **Robotics** – assisting robots in motion estimation and navigation.
-

10. How can Optical Flow improve the performance of surveillance, robotics, and human activity recognition systems?

Optical Flow provides information about the **movement of objects and people across video frames**.

- **Surveillance:** It can track people and vehicles and help detect unusual or abnormal movements.
- **Robotics:** It can help robots estimate their own motion, detect obstacles, and navigate their surroundings.
- **Human Activity Recognition:** It can analyze movement patterns and help recognize activities such as walking, running, waving, and falling.

Therefore, Optical Flow improves systems by providing useful **motion information for tracking, navigation, and activity analysis**.